

УДК 004.056.57

И. А. Беляков**ПРИМЕНЕНИЕ ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ
ПРИ ПОИСКЕ УЯЗВИМОСТЕЙ В ИСХОДНЫХ ТЕКСТАХ
ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ**

Проблема информационной безопасности программного обеспечения (ПО) является актуальной и связана с наличием в нем уязвимостей. В статье рассмотрена проблема поиска уязвимостей в исходных текстах программного обеспечения. Обоснована необходимость использования искусственных нейронных сетей для повышения эффективности поиска уязвимостей.

информационная безопасность, программное обеспечение, уязвимости, сигнатурный поиск, искусственная нейронная сеть.

Введение

Информационная безопасность современной организации во многом определяется защищенностью ее активов, основным из которых является информация. Практически любая крупная компания обрабатывает конфиденциальную информацию, несанкционированный доступ к которой может привести к существенным финансовым потерям.

В настоящее время для обеспечения безопасности информации, как правило, используются дополнительные средства, такие как антивирусы, межсетевые экраны, сканеры безопасности. Недостаточно внимания уделяется безопасности программного обеспечения. Аналитические отчеты крупнейших компаний в области информационной безопасности программного обеспечения показывают, что количество уязвимостей, выявляемых в программном обеспечении, за последние годы существенно увеличилось [1], [2], [3]. В частности, аналитический отчет корпорации IBM показывает актуальность проблемы (рис. 1).

Использование уязвимого, небезопасного программного обеспечения может привести к серьезным последствиям. К наиболее явным относятся несанкционированный доступ к конфиденциальной информации и к информационным системам организации. Аналитическое исследование компании InfoWatch [3] показывает заметный рост количества выявленных утечек конфиденциальной информации.

Наиболее распространенным методом обеспечения безопасности ПО является поиск опасных конструкций в исходных текстах программного обеспечения. Как правило, он осуществляется в соответствии с базой сигнатур, что не является эффективным, так как, используя данный метод,

можно выявить только те уязвимости, сигнатуры которых известны. Следовательно, необходимо создать метод, позволяющий производить более гибкий поиск.

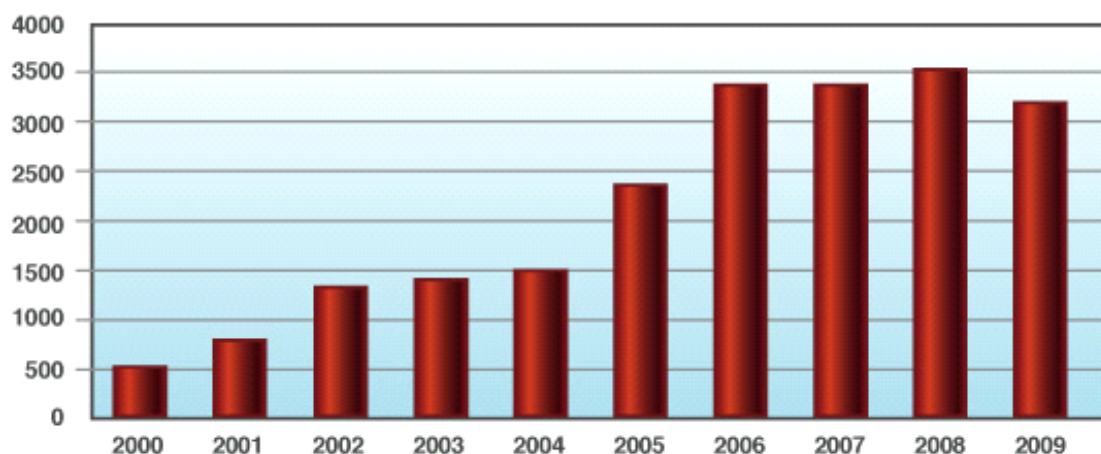


Рис. 1. Количество уязвимостей, выявленных в программном обеспечении

Предложенный в статье подход позволяет выявлять не только те уязвимости, которые полностью соответствуют сигнатуре, но и те, которые в результате искажения не могут быть выявлены существующими методами.

1 Существующий подход к поиску уязвимостей и его недостатки

Одним из распространенных подходов к повышению информационной безопасности ПО является поиск опасных конструкций в исходных текстах. При проведении сертификационных испытаний программного обеспечения по требованиям безопасности в соответствии с руководящим документом ФСТЭК России «Защита от несанкционированного доступа к информации. Часть 1. Программное обеспечение средств защиты информации. Классификация по уровню контроля отсутствия недеklarированных возможностей» одной из базовых проверок ПО является контроль наличия заданных конструкций в исходных текстах. Поиск осуществляется на основе сигнатур.

Сигнатура – характерный признак уязвимости, используемый для ее обнаружения. Большинство современных антивирусов, сканеров уязвимостей и систем обнаружения вторжений используют «синтаксические» сигнатуры, взятые непосредственно из тела атаки (файла вируса или сетевого пакета, принадлежащего эксплойту). Также существуют сигнатуры, основанные на поведении или аномалиях, например слишком агрессивное обращение к какому-либо сетевому порту на компьютере. Пример сигнатуры из тела вируса Email-Worm.Win32.Happy, опубликованный в журнале Virus Bulletin: «Happy New Year 1999 !!»

Сигнатурные методы позволяют описать уязвимость набором правил или с помощью формальной модели, в качестве которой может применяться символьная строка, семантическое выражение на специальном языке и т. п. Суть данного метода заключается в использовании специализированной базы данных шаблонов (сигнатур) атак для поиска действий, подпадающих под определение «атака».

Сигнатурный метод может защитить от вирусной или хакерской атаки, когда уже известна сигнатура уязвимости (например, неизменный фрагмент тела вируса) и она внесена в базу данных. Кроме того, несмотря на кажущуюся простоту сигнатурного метода, и в его реализации есть свои тонкости. Классический пример – с помощью поиска сигнатуры

/../../../../../../../../local.ida

и простого сравнения битовой информации невозможно выявить хакерскую атаку на HTTP-сервер. Нападающий может легко изменить строку в соответствии с соглашением об URI и использовать битовую строку %2F%2E%2F%2E%2F%2E%2F%2E%2F%2E%2F%2E%2F%2E%2F%2E%2F%2Elocal.ida, которую данная сигнатура уже не охватывает [2].

Таким образом, эффективность работы сигнатурной системы выявления уязвимостей определяется тремя основными факторами: оперативностью пополнения сигнатурной базы, ее полнотой с точки зрения определения сигнатур атак, а также наличием эффективных алгоритмов сведения действий атакующих к некоторым базовым шагам, в рамках которых происходит сравнение с сигнатурами.

2 Использование искусственных нейронных сетей для поиска уязвимостей

Решение задачи поиска заданных конструкций относится к классу задач типа «классификация объекта». Наиболее эффективной ИНС для решения поставленной задачи является многослойная ИНС прямого распространения [8]. Модель искусственной нейронной сети (ИНС), решающей задачу интеллектуального поиска заданных конструкций в исходных текстах ПО, была разработана средствами программного комплекса Matlab 7.

Выход нейрона определяется как функция его состояния:

$$y = f(s). \quad (1)$$

Эта функция называется активационной или передаточной. Она может быть различной (в том числе нелинейной) и ее вид определяет функционирование нейрона, а также диапазон значений выходного сигнала. В линейных сетях эта функция есть само состояние, т. е. нейрон линейной сети подает свое состояние на выход без изменений:

$$f_s = s = \sum_i \omega_i x_i. \quad (2)$$

Нейронная сеть состоит из слоев нейронов. Обычно нейроны одного слоя не связаны между собой, а их выходы являются входами для нейронов следующего слоя [7]. Исходя из начальной постановки задачи выявления заданных конструкций, для обучения ИНС необходимо использовать метод обратного распространения ошибки, который направлен на минимизацию разницы между реальным и ожидаемым выходами сети за счет изменения весов синапсов. Пусть у нас есть P пар входных и выходных образов $((\bar{x}_1, \bar{t}_1), (\bar{x}_2, \bar{t}_2), \dots, (\bar{x}_p, \bar{t}_p))$, $\bar{x}_p = (x_1^p, x_2^p, \dots, x_n^p)$, $\bar{t}_p = (t_1^p, t_2^p, \dots, t_m^p)$, тогда величина, характеризующая расхождение реального выхода с ожидаемым,

$$E = \frac{1}{2} \sum_p \sum_k (t_k^p - y_k^p)^2. \quad (3)$$

Здесь t_k^p – k -е значение в p -м выходном образе; y_k^p – k -е значение в реальных выходных данных, полученных при подаче на вход p -го входного образа $\bar{x}_p$ ($k = 1, 2, \dots, m$). Суммирование ведется по всем входным последовательностям $\bar{x}_p$, т. е. сначала через сеть пропускают все наборы входных сигналов $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_p$, получая при этом наборы выходных сигналов $\bar{y}_1, \bar{y}_2, \dots, \bar{y}_p$, а затем вычисляют E . Метод обратного распространения ошибки заключается в изменении весов связей каждого слоя на величины, определяемые через полученное значение E .

Для создания объекта, представляющего ИНС, можно использовать общую функцию:

$$\langle \text{сеть} \rangle = \text{newff}(\text{PR}, [\text{S1 S2} \dots \text{SN}], [\text{T1 T2} \dots \text{TN}], \text{F}),$$

где PR – матрица размером $k \times 2$ – задает диапазоны входных значений; [S1 S2 ... SN] – количество нейронов в слое 1, 2, ... N; [T1 T2 ... TN] – активационная функция для слоя 1, 2, ..., N; F – функция обучения, применяемая для этой сети [9].

При этом в качестве обучающего алгоритма выбран алгоритм Levenberg-Marquardt (trainlm). Этот алгоритм обеспечивает быстрое обучение, но требует много ресурсов. В случае, если для реализации этого алгоритма не хватит оперативной памяти, можно использовать другие алгоритмы (trainbfg, trainrp, trainscg, traincgb, traincgf, traincgp, trainoss, traingdx). По умолчанию используется trainlm. Модель ИНС, разработанной для поиска узвмостей, представлена на рисунке 2.

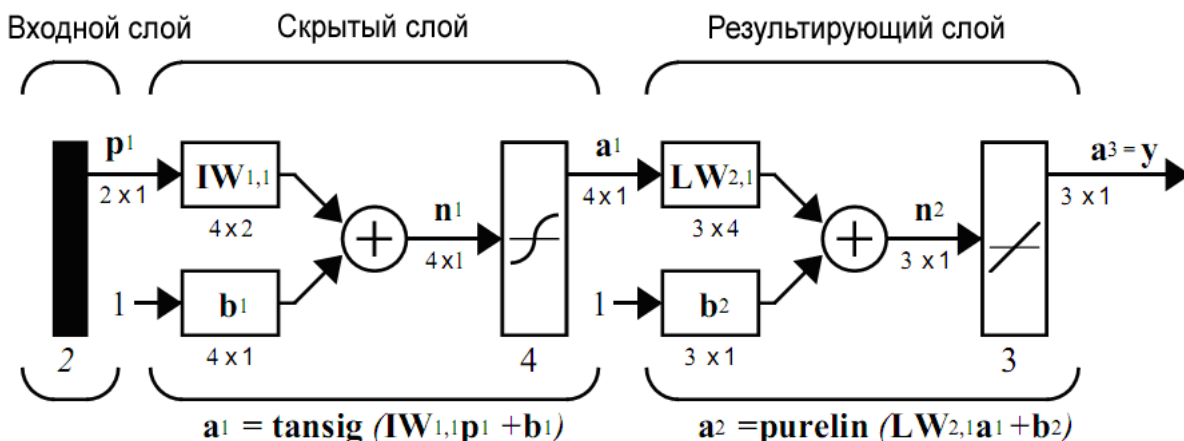


Рис. 2. Модель ИНС для поиска уязвимостей

В ходе тестирования ИНС решалась задача поиска участков кода, в которых используется переменная для хранения пароля. Подобные переменные достаточно легко могут быть выявлены злоумышленником, что может привести к несанкционированному доступу в систему. Моделирование ИНС средствами MatLab накладывает ограничения, существенные для решения задачи поиска заданных конструкций. Для повышения эффективности работы ИНС поиск уязвимостей предлагается осуществлять по кодам символов в соответствии с таблицей ASCII. Пример обучающей выборки представлен в таблице 1.

ТАБЛИЦА 1. Параметры обучения нейронной сети

Параметр	Обучающая последовательность																	
	В формате текста									В закодированном формате								
P(1):	p	p	p	p	w	n	=	q	a	112	112	112	112	119	110	61	113	97
P(2):	a	a	a	s	p	b	=	a	s	97	97	97	115	112	98	61	97	115
P(3):	s	s	s	w	o	g	r	z	d	115	115	115	119	111	103	114	122	100
P(4):	s	s	w	d	r	h	t		r	115	115	119	100	114	104	116	32	114
P(5):	w	q	d		w	j	g	=	t	119	113	100	32	119	106	103	61	116
P(6):	o	o		=	o	k		p	f	111	111	32	61	111	107	32	112	102
P(7):	r	p	=		r	l	=	p	o	114	112	61	32	114	108	61	112	111
P(8):	d	v	t	t	d	w	r	s	p	100	118	116	116	100	119	114	115	112
P(9):		=	h	e	=	e	l	q	l	32	61	104	101	61	101	108	113	108
P(10):	=	w	l	q	y	z	p	e	z	61	119	108	113	121	122	112	101	122
T(1):	1	1	1	1	0	0	0	0	0	1	1	1	1	0	0	0	0	0
T(2):	0	0	0	0	1	1	1	1	1	0	0	0	0	1	1	1	1	1

Диапазон входных значений для входного слоя нейронов определяется крайними значениями таблицы ASCII = [1, 255], размерность анализируемого вектора $P\{\}$ для тестового примера равна 10 и определяется количеством символов в строке исходных текстов, которая может содержать

«опасную» переменную. Результирующий вектор $T\{\}$ состоит из двух элементов: значение $T(1)$ говорит о наличии уязвимости, а значение $T(2)$ – о ее отсутствии.

Для решения поставленной задачи сформируем трехслойную сеть прямого распространения ($N = 3$), включающую 10 нейронов во входном слое (по числу компонент входного вектора P) с передаточной функцией `logsig`. Исходя из условий решаемой задачи, диапазон входных значений для каждого нейрона входного слоя будет лежать в пределах от 1 до 255. Внутренний (второй) слой будет содержать 100 нейронов с передаточной функцией `logsig` и 2 нейрона в выходном слое с передаточной функцией `purelin` (значение первого нейрона будет показывать, что входной вектор содержит заданную конструкцию, а значение второго нейрона будет показывать, что заданная конструкция отсутствует).

Пример:

```
>> P = [112 112 112 112 119 110 61 113 97; 97 97 97 115 112 98 61 97 115;  
115 115 115 119 111 103 114 122 100; 115 115 119 100 114 104 116 32 114;  
119 113 100 32 119 106 103 61 116; 111 111 32 61 111 107 32 112 102; 114  
112 61 32 114 108 61 112 111; 100 118 116 116 100 119 114 115 112; 32 61  
104 101 61 101 108 113 108; 61 119 108 113 121 122 112 101 122]  
>> T = [1 1 1 1 0 0 0 0 0; 0 0 0 0 1 1 1 1 1]  
>> net=newff([1 255; 1 255; 1 255; 1 255; 1 255; 1 255; 1 255; 1 255; 1 255; 1  
255],[10,100,2],{'logsig' 'logsig' 'purelin'},'trainlm');  
net.performFcn='sse';  
net.trainParam.goal=0.01;  
net.trainParam.epochs=1000;  
net=train(net,P,T);
```

Результаты обучения нейронной сети показывают, что время, необходимое для обучения нейронной сети выявлять заданную последовательность длиной 8 символов в векторе длиной 10 символов, занимает 5 секунд и 7 циклов обучения при условии, что обучающий вектор состоит из 5 элементов.

После создания и обучения ИНС необходимо убедиться в ее работоспособности. Для этого проанализируем результаты работы модели в четырех различных ситуациях, когда на вход нейронной сети будет подаваться вектор (табл. 2), содержащий:

- сигнатуру поиска;
- искаженную (на 12 %, 25 %, 37 % и 50 %) сигнатуру поиска;
- произвольную последовательность символов.

ТАБЛИЦА 2. Тестовая последовательность для проверки ИНС

Тестовая последовательность									
Символьная					Цифровая				
p	p	p	q	g	112	112	112	113	103
a	a	a	s	f	97	97	97	115	102
s	s	s	h	o	115	115	115	104	111
s	s	u	g	r	115	115	117	103	114
w	w	s	f	s	119	119	115	102	115
o	o		b	a	111	111	32	98	97
r	p	=	j	r	114	112	61	106	114
d	d	t	k	f	100	100	116	107	102
		h	l	h	32	32	104	108	104
=	=	l	a	j	61	61	108	97	106

Проверка

```
>> Tr = [112 112 112 113 103; 97 97 97 115 102; 115 115 115 104 111; 115
115 100 103 114; 119 119 32 102 115; 111 111 32 98 97; 114 112 61 106 114;
100 100 116 107 102; 32 32 104 108 104; 61 61 108 97 106]
```

```
>> res = sim(net,Tr)
```

```
res =
```

```
1.0198 1.0215 0.9882 0.0090 -0.0139
-0.0082 -0.0089 -0.1569 0.9757 1.0330
```

Полученная в результате матрица *res* показывает, что нейронная сеть обнаружила уязвимости в первой и второй тестовых последовательностях (первый и второй столбцы матрицы *Tr*), а также отсутствие в третьей, четвертой и пятой тестовых последовательностях (третий, четвертый и пятый столбцы матрицы *Tr*), что в полной мере соответствует действительности.

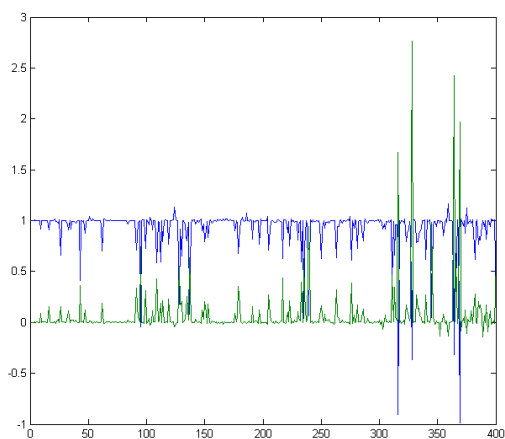
Для оценки степени влияния искажения вредоносной последовательности на результат работы ИНС проведем ее обучение на выявление одной сигнатуры (8 символов) в входном векторе, состоящем из 10 элементов. В ходе проверки смоделируем ситуации, в которых вредоносная последовательность будет искажаться на 12,5 %, 25 %, 37,5 % и 50 %. Для каждой ситуации проведем 100 проверок, в ходе которых анализируемый вектор в соответствии со степенью искажения будет заполняться случайными силами и анализироваться. Полученные результаты показывают, что ИНС позволяет выявлять вредоносную последовательность (рис. 3, а). При этом следует отметить, что при увеличении искажения количество ошибок становится существенным (табл. 3).

ТАБЛИЦА 3. Значения вероятностей ошибок первого и второго рода

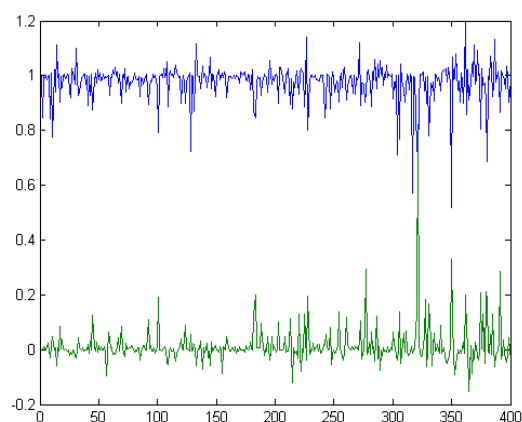
Оцениваемая характеристика	Искажение сигнатуры			
	0,125	0,25	0,375	0,5
Количество проверок	100	100	100	100
Вероятность ошибок первого рода	1 %	4 %	19 %	41 %
Вероятность ошибок второго рода	0 %	0 %	1 %	3 %

Увеличение количества сигнатур, участвующих в процессе обучения нейронной сети, не оказывает существенного влияния на время, требуемое на обучение, но при этом увеличивается количество уязвимостей, выявляемых нейронной сетью (рис. 3). Увеличение количества выявляемых уязвимостей не оказывает влияния на количество ошибок первого и второго рода (табл. 4). Следует отметить, что при этом снижается значимость каждого отдельного байта в обучающей последовательности, что положительным образом сказывается на устойчивости работы нейронной сети.

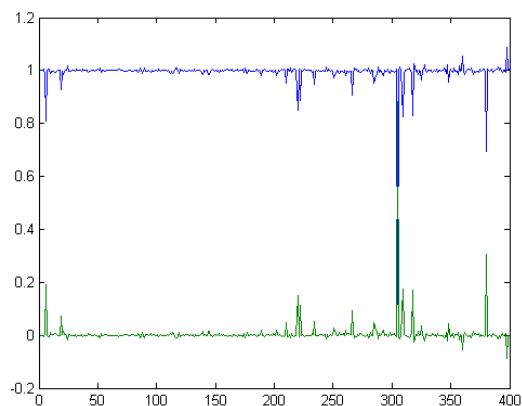
а)



б)



в)



г)

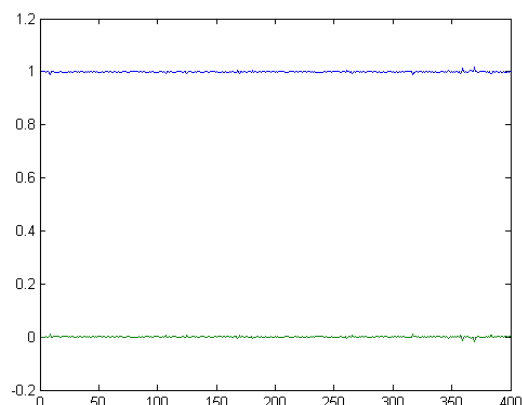


Рис. 3. Результаты работы ИНС для одной (а), двух (б), пяти (в) и десяти (г) сигнатур в условиях их искажения

Полученные результаты наглядно показывают эффективность применения ИНС для решения задач выявления вредоносных последовательностей в условиях их искажения.

ТАБЛИЦА 4. Оценка результатов работы ИНС

Искажение	Количество выявляемых сигнатур			
	1	2	5	10
Вероятность выявления уязвимости				
12,5 %	0,9645	0,9811	0,9972	0,9994
25 %	0,9458	0,9852	0,9990	0,9993
37,5 %	0,9346	0,9808	0,9928	0,9994
50 %	0,8622	0,9623	0,9860	0,9997
Вероятность ошибки первого рода				
12,5 %	0,0307	0,0052	0,0026	0,0005
25 %	0,0491	0,0071	0,0007	0,0006
37,5 %	0,0592	0,0144	0,0070	0,0005
50 %	0,1457	0,0258	0,0137	0,0002

Результаты тестирования ИНС показывают, что незначительные отклонения от заданной последовательности не оказывают существенного влияния на их выявление. Следовательно, можно сделать вывод, что использование ИНС для выявления заданных конструкций является более результативным решением, чем поиск по базе сигнатур, так как позволяет выявлять не только те уязвимости, сигнатуры которых есть в базе сигнатур, но и все подозрительные участки кода.

Заключение

Предложенный подход позволяет существенно повысить эффективность поиска уязвимостей и опасных конструкций в условиях искажения вредоносной последовательности. Принимая во внимание преимущества использования ИНС, следует отметить, что создание и обучение ИНС является более сложной задачей, чем сбор сигнатур. Также при анализе исходных текстов с помощью ИНС производится существенно больше математических вычислений, поэтому скорость анализа напрямую зависит от производительности используемых аппаратных средств.

Таким образом, использование интеллектуальных средств является перспективным направлением для решения задач выявления уязвимостей вне зависимости от применяемых технологий их сокрытия.

Список использованной литературы

1. **IBM X-Force** Threat Analysis / Аналитический отчет корпорации IBM за 2009 г. – <http://www-935.ibm.com/services/us/index.wss/offering/iss/a1026943>.
2. **OWASP** Top 10 – 2010 rc1 / Аналитический отчет консорциума OWASP за 2010 г. – 21 с. – http://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project.

3. **InfoWatch** Global Data Leakage Report / Infowatch. – 11 с. – www.infowatch.ru, 2010.
4. **Безопасность** программного обеспечения компьютерных систем / О. В. Казарин. – М. : МГУИ, 2003. – 213 с. – ISBN 5-283-01667.
5. **Home** Network Security Simplified. How Antivirus Works / J. Doherty, N. Anderson. – Cisco Press, 2006. – 224 с. – ISBN 1-58720-163-1.
6. **Kalman** filtering and neural networks / Simon Haykin. – Wiley-interscience publication, 2001. – 202 p. – ISBN 0-471-22154-6.
7. **Основные** концепции нейронных сетей / Роберт Калан. – М. : Изд. дом «Вильямс», 2001. – 291 с. – ISBN 5-8459-0210-X.
8. **Нейронные** сети для обработки информации / С. Осоровский. – М. : Финансы и статистика, 2002. – 344 с. – ISBN 5-279-02567-4.
9. **Нейронные** сети. MATLAB 6 / В. С. Медведев, В. Г. Потемкин. – М. : Диалог-МИФИ, 2002. – 496 с. – ISBN 5-86404-163-7.
10. **MATLAB** для студента / А. М. Половко, П. Н. Бутусов. – СПб. : БХВ-Петербург, 2005. – 320 с. – ISBN 5-94157-595-5.
11. **Тестирование** объектно-ориентированного программного обеспечения : практическое пособие / Макгрегор Джон, Сайкс Девид. – Киев : ТИД ДС, 2002. – 432 с. – ISBN 432 966-7992-12-8.
12. **Интеллектуальные** системы поддержки принятия решений в нештатных ситуациях с использованием информации о состоянии природной среды / В. А. Геловани, А. А. Башлыков, В. Б. Бритков, Е. Д. Вязилов. – М. : Эдиториал УРСС, 2001. – 304 с. – ISBN 304 5-8360-0298-3.

*Статья поступила в редакцию 22.10.2010;
представлена к публикации членом редколлегии А. А. Корниенко.*

УДК 539.3

А. В. Бенин

МОДЕЛИРОВАНИЕ ПРОЦЕССОВ ДЕФОРМИРОВАНИЯ И РАЗРУШЕНИЯ ПРИ ВЫТЯГИВАНИИ АРМАТУРНОГО СТЕРЖНЯ ИЗ БЕТОННОГО БЛОКА

Задача о вытягивании арматурного стержня из бетонного блока является актуальной для практики, так как представляет самый распространенный способ экспериментального определения характеристик сцепления арматуры с бетоном, необходимых для оценки прочности и деформационной способности железобетонных конструкций. В рамках данного исследования получен ряд нелинейных конечно-элементных решений задачи о вытягивании арматуры из бетонного блока на основе различных подходов учета явления сцепления. Произведено сравнение полученных решений с экспериментом.

сцепление арматуры с бетоном, конечно-элементное моделирование, математические модели.